



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 – Estructuras de Datos y Algoritmos

2026 - 1

Taller 5

Objetivos

- Utilizar la estrategia algorítmica de Backtracking.
- Diseñar optimizaciones para algoritmos de Backtracking.

Introducción

En la región de Kanto, los entrenadores que realizan su viaje por la región deben cruzar el famoso Túnel Roca. Para recorrerlo, además de los molestos Zubat y los fuertes Machop, tendrán que lidiar con la falta de iluminación dentro de esta zona. Esto puede solucionarse enseñando a un pokémon el movimiento Destello, que permite iluminar la cueva, sin embargo a muchos entrenadores se les olvida obtener la MT que se requiere para aprenderlo.



El profesor Oak ha escuchado esta historia repetirse incontables veces, con lo que decide que, para ayudar a estos entrenadores a encontrar un camino en la oscuridad, implementará una nueva funcionalidad en la Pokédex.

Por tanto, tu misión esta vez será diseñar un algoritmo que sea capaz de decirle a los entrenadores si existe un camino libre de obstáculos entre dos puntos dentro del túnel, para que puedan navegarlo en la oscuridad.

¡Cruzando Túnel Roca en la Oscuridad!



Problema

Dentro de Kanto, incontables entrenadores olvidadizos deben cruzar el Túnel Roca a ciegas, lo que resulta muy demoroso y peligroso. Por tanto, el profesor Oak decide implementar una funcionalidad en la Pokédex que permita a los entrenadores encontrar un camino en la oscuridad dentro del túnel.

Para ello, debes implementar un algoritmo que, dada una matriz que representa una porción dentro del túnel, permita determinar si existe un camino libre de obstáculos entre dos puntos de dicha matriz.

Entidades

Matriz: Cada matriz que se te entregará tendrá dimensiones del tipo $M \times M$, es decir, serán matrices cuadradas. Además, cada valor dentro de la matriz será un número entero, donde 0 representa una casilla libre, y 1 una casilla con un obstáculo.

Flujo

Primero recibirás un entero N que indica la cantidad de eventos a procesar.

Luego de ello, recibirás N eventos, donde cada uno representa una nueva operación de búsqueda de camino. Estos deben ser procesados en el orden en que aparecen.

Parte Formativa - Eventos

FIND-PATH {M} {K}

En este evento, dada una matriz de dimensiones $M \times M$, deberás determinar si es posible encontrar un camino válido de tamaño K desde la esquina superior izquierda, hasta la esquina inferior derecha de la matriz, es decir, entre las casillas $(0, 0)$ y $(M - 1, M - 1)$. Cabe destacar que los contenidos de la matriz se dan en las M líneas de input posteriores a la línea del evento en sí.

a) Implemente el evento FIND-PATH

Su tarea para esta parte es implementar en C las siguientes dos funciones:

1. `bool check(int** matrix, int M, int K, int i, int j)`: Esta función verifica que visitar la casilla (i, j) sea válido, retornando `true` en dicho caso, y `false` en caso contrario. Para ello, debes revisar que las siguientes cuatro condiciones se cumplan:

- (a) La casilla (i, j) está dentro de la matriz.
- (b) La casilla (i, j) no tiene obstáculos (es decir, que tenga un valor distinto a 1).
- (c) La casilla (i, j) no ha sido visitada anteriormente (puedes usar cualquier valor distinto de 0 y 1 para marcar una casilla visitada).
- (d) La cantidad de pasos restantes K es mayor a cero.

Si todas estas condiciones se cumplen simultáneamente, se considera que visitar la casilla (i, j) es válido, y por tanto `check` debe retornar `true`. En otro caso, se considera inválido visitar esta casilla, y por tanto la función retorna `false`.

2. `bool find_path(int** matrix, int M, int K, int i, int j)`: Esta función analiza si existe un camino válido de largo K entre la casilla $(0, 0)$ y $(M - 1, M - 1)$, retornando `true` en dicho caso, y `false` en caso contrario. Para ello, debes realizar los siguientes pasos:

- (a) Retornar `true` si la casilla actual (i, j) es la casilla $(M - 1, M - 1)$ y el valor de K es cero.
- (b) Llamar a la función `check` y revisar su valor
- (c) En caso de que esta retorne `false`, retornar `false`
- (d) Si el retorno del `check` es `true`, marcamos la casilla (i, j) como visitada (usando cualquier entero distinto de 0 y 1)
- (e) Luego hacemos un llamado recursivo para cada casilla adyacente, es decir, las casillas una posición arriba, abajo, a la izquierda y a la derecha de la actual, con el valor de K disminuido en uno.
- (f) Si alguno de los llamados recursivos anteriores, retorna `true` retornar `true`.
- (g) En caso de que ninguno retorne `true`, marcamos la casilla actual como libre (volvemos a setear su valor como cero).

b) Diseñe una optimización para FIND-PATH

Dado el algoritmo implementado en la parte (a), proponga una optimización que mejore su eficiencia, utilizando las técnicas de mejora para Backtracking vistas en clase.

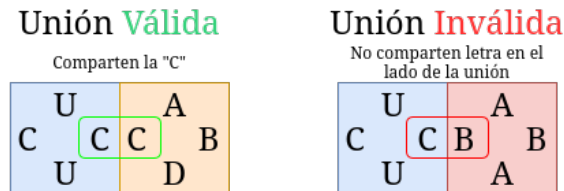
Explique qué técnica utilizó, cómo mejora el rendimiento del algoritmo de la parte (a), y escriba el pseudocódigo resultante al usar la técnica escogida.

Parte Evaluada - Eventos

Para esta parte, se te entregarán baldosas cuadradas que tienen una letra en cada lado, es decir, una arriba, una abajo, una a la izquierda y una a la derecha. En el código, esto se implementó con el siguiente `struct`:

```
typedef struct {
    char up;
    char down;
    char left;
    char right;
} Tile;
```

Estas poseen la siguiente regla: para poder colocar una baldosa en una matriz, se debe cumplir que este adyacente a otras baldosas cuyos lados colindantes coincidan en color, al igual que un dominó. Por ejemplo:



Por tanto, el problema a resolver para esta parte será, dado un conjunto de estas baldosas, donde puedes usar una cantidad ilimitada de cada una de ellas, verificar si es posible rellenar por completo una matriz vacía de dimensiones $M \times M$.

a) Implemente el evento `TILING {M} {T}`

En este evento, dada una matriz vacía de dimensiones $M \times M$, deberás determinar si con el conjunto de baldosas dado de tamaño T , es posible embaldosar por completo la matriz. Para ello, tienes usos ilimitados de cada tipo de baldosa, y debes respetar la regla para colocar nuevas baldosas descrita anteriormente.

Cabe destacar que, dado que la matriz está vacía, sólo se entrega su dimensión M . Además, en las T líneas de input posteriores al evento como tal, se encuentra cada tipo de baldosa que tendrás disponible.

Para este evento deberás implementar las siguientes dos funciones en C:

1. `bool check_tile(Tile** matrix, int k, int l, Tile tile):`

Esta función verifica que colocar la baldosa `tile` en la casilla (k, l) sea válido. Para ello, debes retornar `false` si alguna de las siguientes condiciones no se cumple y `true` en caso contrario:

- (a) Si $k > 0$ y el color de **abajo** de la baldosa en la casilla $(k-1, l)$ coincide con el color de **arriba** de `tile`
- (b) Si $l > 0$ y el color de la **derecha** de la baldosa en la casilla $(k, l-1)$ coincide con el color de la **izquierda** de `tile`

2. `bool tiling(Tile** matrix, int M, Tile available_tiles[], int T, int k, int l):`

Esta función analiza si es posible embaldosar la matriz `matrix` de dimensiones $M \times M$, utilizando el conjunto de baldosas `available_tiles` de tamaño `T`, retornando `true` en dicho caso, y `false` en caso contrario. Para completar la función, debes realizar los siguientes pasos:

- (a) Retornar `true` si el valor de `k` es igual a `M`.
- (b) Guardar en una nueva variable `next_k` el valor de `k`, y en una nueva variable `next_l` el valor `l + 1`.
- (c) Comparar `next_l` con `M`. Si son iguales, actualizar el valor de `next_l` a 0 y el de `next_k` a `k + 1`.
- (d) Iterar los siguientes pasos, iniciando una variable `i` desde 0 hasta `T - 1`, incrementando su valor en 1 por ciclo:
 - i. Llamar a la función `check_tile` con la `i`-ésima baldosa de `available_tiles`.
 - ii. Si el retorno de `check_tile` es `false`, no hacemos nada y seguimos con la siguiente iteración.
 - iii. Si el retorno del check es `true`, primero asignamos a la casilla `(k, l)` la `i`-ésima baldosa de `available_tiles`.
 - iv. Luego, hacemos un llamado recursivo a la función `tiling` pero ahora usando los valores de `next_k` y `next_l`.
 - v. En caso de que el llamado recursivo anterior retorne `true`, retornamos `true`.
- (e) Una vez finalizado el bucle del paso (d), retornar `false`.

b) Sección Teórica: TILING-SOLVER {M} {T}

En este evento, dada una matriz vacía de dimensiones $M \times M$ y un conjunto de baldosas de tamaño `T`, debes determinar si existen cero, uno o más de un embaldosado válido posible. Para ello, del mismo modo que en el evento anterior, tienes usos ilimitados para cada tipo de baldosa, y debes respetar la regla para colocar nuevas baldosas descrita al comienzo. Además de esto, en el caso de que exista más de un embaldosado posible, solamente debes indicar esto, sin dar el número exacto de soluciones válidas posibles, por lo tanto basta que con que tu algoritmo retorne ints 0, 1 ó 2 para representar los casos donde no hay soluciones, donde hay una única solución o donde hay más de una solución, respectivamente.

Para este evento deberás responder las siguientes preguntas:

1. ¿Cómo cambiarías el backtracking de la parte A para cumplir con las nuevas condiciones del problema?
2. Para el nuevo backtracking, proponga una optimización para mejorar la eficiencia del algoritmo, usando alguna de las técnicas vistas en clase (poda, heurística o propagación).

Para la primera pregunta, **explique su solución y proponga el pseudocódigo que implementa dicha solución**. Para la segunda basta con explicar la mejora.

Bonus

Implemente en el código su solución para las dos preguntas anteriores, es decir, el evento y la optimización propuesta.

Input

El archivo de input comenzará con un número N que corresponde a la cantidad de eventos a recibir. Luego, se entregarán los N eventos, cada uno con la información necesaria para realizar una nueva operación de verificación de embaldosado.

Como ejemplo tenemos el siguiente input:

input	
1	3
2	TILING 5 1
3	A A A A
4	TILING 4 2
5	A A I D
6	I D A A
7	TILING-SOLVER 4 2
8	A A B B
9	B B A A

Output

output	
1	Se encontro tiling con las piezas.
2	No se encontro tiling con las piezas.
3	Existe mas de un tiling posible con las piezas.

Evaluación (Solo parte evaluada)

La nota de tu taller es calculada a partir de test cases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán test cases públicos para que puedas probar tu implementación durante el taller.

Además, para la parte teórica del evaluado, se espera una respuesta en sus propias palabras que se evaluara de manera manual. Por lo que se espera que el estudiante pueda explicar con sus propias palabras cómo resolvería el problema, y que proponga el pseudocódigo que implemente dicha solución.

Finalmente el Bonus también se evaluará con test cases.

Integridad académica e IA

Cualquier acto deshonesto o fraude académico está estrictamente prohibido. Se prohíbe el uso de herramientas de inteligencia artificial y el uso o manipulación de otros dispositivos electrónicos como laptops, tablets, y/o teléfonos durante la evaluación.

Proceso de Entrega

Conexión a internet

Una vez se haya iniciado sesión en el escritorio, seleccionar el ícono de redes arriba a la derecha. Luego: *"avaiablenetwork"* → *"eduroam"*. Para conectarse a eduroam, deben tener la siguiente configuración:

- Tener seleccionada la opción "No CA certificate is required".
- En la opción de *"Inner authentication"* seleccionar *"PAP"*
- Luego debe ingresar tus credenciales usuario UC. **IMPORTANTE:** en el usuario debes incluir el @, no sólo el nombre.
- Ejemplos correctos: *"nombre@uc.cl"*; *"nombre@estudiante.uc.cl"*, Incorrecto: "nombre" sin el @.

Descargar taller

Para descargar el código base que con el que trabajarán, deben abrir una terminal, estar conectados a internet y ejecutar `descargar-taller nombre_archivo.zip`

Donde el *nombre_archivo* debe ser reemplazado con el nombre que se avisará al inicio de cada taller.

Descomprimir archivo

Una vez descargado el archivo del taller deben descomprimirlo usando: `unzip nombre_archivo.zip`

Editor de código

El editor de código del entorno se llama *"lite - xl"*. Para abrir su trabajo tienen varias alternativas, las recomendables son:

- Abrir el editor de código, buscar la sección de *"file"* y seleccionar *"Open"* y buscar la carpeta donde quieres trabajar.
- Ir al explorador de archivos, buscar la carpeta donde quieres trabajar, hacer click derecho en la carpeta y seleccionar *"Openwith..."* y seleccionar el editor.

- **IMPORTANTE:** El editor **no tiene autoguardado**, por lo que cada vez que hagan cambios deben usar "*CTRL + S*" para guardar. Si ustedes no guardan sus archivos e intentan compilar, el proceso de compilado no guarda automáticamente, por lo que su código se compilará con la última versión que hayan guardado, **DEBEN GUARDAR ANTES DE COMPILAR**.

Espacio de trabajo

Al abrir el explorador de archivos o una terminal, verán una carpeta llamada código. Es **FUNDAMENTAL** que todo el trabajo que hagan lo tengan **dentro de esa carpeta**. Eso les asegura que en caso de que el entorno falle y tengan que reiniciar el computador, todo lo que está guardado en esa carpeta se mantendrá.

Uso del entorno de trabajo

En su carpeta de trabajo tendrán al menos los siguientes elementos:

- **src/**: carpeta con su código
- **Makefile**: archivo para compilar
- **tests/**: carpetas de tests públicos
- **run.sh**: script para ejecutar todos los tests

Para compilar, en una terminal usen:

```
make
```

Para usar el script que ejecuta todos los tests, usen:

```
./run.sh
```

El run.sh genera una carpeta llamada **outputs** que guarda para cada test, el output de su programa. **Si desean ejecutar un único test**, ocupen:

```
./ejecutable ruta_al_input_del_test.txt archivo_output.txt
```

Donde ejecutable es el nombre del archivo que se genera al compilar, el cual puedes revisar en las carpetas dentro de **src**. Ejemplos de ejecutables: "pokemones", "magnemites". Ejemplo de ruta al archivo de input: **tests/F1/pokemones/easy-0/input.txt** y **output.txt** es un archivo txt que crean y es donde irá su salida.

En caso de querer debugear algún error de un test en específico usar **valgrind**

```
valgrind ./ejecutable ruta_a_input.txt output.txt
```

Entrega Taller

Para entregar, también es importante que estén conectados a internet, entrar en una terminal, posicionarse en la ruta donde está la carpeta de su entrega (no dentro de la carpeta misma). Finalmente, ejecutar

```
entregar-taller nombre_carpeta TX nAlumno1_nAlumno2_...
```

Donde:

- **nombre_carpeta** es el nombre de la carpeta que quieren entregar. Por ejemplo: **T2_evaluado_G2**.
- **TX**, donde X corresponde al número del taller que se está entregando. Por ejemplo: **T2**.
- **nAlumno1_nAlumno2_...** son los números de alumno de los integrantes del grupo que trabajaron, separados por "_" (Si están realizando este taller de manera solitaria, solamente escriben su número)

de alumno).

Ejemplos Correctos:

12345678_23456789 20387645 20387645_12345678_23456789

Ejemplos Incorrectos:

1234567823456789 *Nombre1_12345678* *Nombre1_Nombre2_Nombre3*